Action Directives — Administrator Manual

Revision: 2 **Last modified:** 2026-07-14T22:05:00Z **Authority:** constitution §11.4.140 (action-prefix system) + §11.4.202 (reporting directives) + §11.4.164 (post-update auto-propagation) + §11.4.75 (mechanical enforcement) **Maintainer:** constitution submodule (inherited BY REFERENCE per §11.4.28(B) / §11.4.177) **Scope:** for whoever wires the directive system into a consuming project — install, verify, auto-load, extend, troubleshoot, uninstall.

Companion documents: REFERENCE.md · USER_GUIDE.md · QUICKSTART_INSTALL.md [1]

. Install

From the **consuming project's root**, with the constitution submodule present — **one command wires everything**:

```
bash constitution/scripts/install_cli_agent_plugins.sh
```

Idempotent. Re-running is always safe. Real output:

```
[install-plugins] regenerated slash commands from registry (10 Claude Code command file(s))
[install-plugins] wired 10 gemini command(s) → .gemini/commands/
[install-plugins] wired 10 qwen command(s) → .qwen/commands/
[install-plugins] wired 10 codex command(s) → prompts/
[install-plugins] wired the UserPromptSubmit prefix hook (free-form 'NAME :: task' forms)
[install-plugins] linked 3 skill(s) into .claude/skills/ (4 legacy dir(s) without SKILL.md skipped)
[install-plugins] plugin 'helix@helix-constitution' is INSTALLED (verified via `claude plugin list`)
[install-plugins] done — action directives + skills are wired for this project
```

It does five things:

1. **Regenerates** every registered action's slash command for every supported agent, from `actions/registry.yaml` (`scripts/generate_agent_prefix_commands.sh`).
2. **Symlinks the other agents' commands** into their project-level command directories — `.gemini/commands/` , `.qwen/commands/` , `prompts/` (Codex). BY REFERENCE (symlink, never a copy); it **never clobbers a real file** the operator wrote by hand — only its own symlink or a free slot (§4).
3. **Wires the `UserPromptSubmit` prompt-expansion hook** by chaining `install_action_prefix.sh` — so the free-form forms (`BUG: …` , `BUG :: …` , `BUG ---> …`) work, not just the slash commands. Registered **exactly once** however many times you re-run (verified: `grep -c action_prefix_expand .claude/settings.json` → `1` after three consecutive installs; two would double-expand every prompt).
4. **Symlinks every constitution skill that has a `SKILL.md`** into `<project>/.claude/skills/<name>` — again BY REFERENCE (§11.4.28 / §11.4.80).
5. **Registers the local marketplace and installs the `helix` plugin** via the `claude plugin` CLI, then **verifies** the result with `claude plugin list` . It never trusts the tool's own success message (§11.4.6 / §11.4.200).

You do **not** need to run `install_action_prefix.sh` separately — step 3 chains it. (It remains the single *owner* of that hook; see §3.)

Flags

Flag	Effect
`--check`	Report state (constitution side **and** project side), change nothing. Exit 0 = fully wired.
`--skills-only`	Link skills only; skip generation, the other agents' commands, the prompt hook, and the plugin install.
`--skill NAME`	Link exactly one skill (implies `--skills-only`). This is what each skill's own `register.sh` calls.
`--no-plugin`	Skip the marketplace/plugin step (useful on a host with no `claude` CLI).
`[PROJECT_ROOT]`	Positional. Defaults to `$PWD`.

2

Verify

```
bash constitution/scripts/install_cli_agent_plugins.sh --check
```

Real output on a fully-wired tree:

```
[install-plugins] constitution side: marketplace + plugin manifests
present; 10 command(s); 3 skill(s)
[install-plugins] project side: 3 skill symlink(s) in .claude/skills/
[install-plugins] project side: UserPromptSubmit prefix hook is registered
[install-plugins] project side: plugin 'helix' is installed + enabled
[install-plugins] VERDICT: WIRED
```

`10 command(s)` = 5 registered actions × 2 files each (`<name>.md` + `default-<name>.md`). `3 skill(s)` = the three canonical Agent Skills.

`--check` verifies **both sides** and exits non-zero on any gap:

Side	Checked
consti-tution	`marketplace.json`, `plugin.json`, ≥1 generated command, ≥1 `SKILL.md`
project	skill symlink count in `.claude/skills/`, the `UserPromptSubmit` hook entry in `.claude/settings.json`, the plugin installed + enabled

Honest boundary (§11.4.6). When the `claude` CLI is **absent**, the plugin state is reported as **UNKNOWN** — not as installed, not as missing. The check never claims either way about a state it cannot resolve.
[3]

. The two installers — one entry point, two owners

You only ever run **one** command (§1). But the split underneath is real and load-bearing — keep it.

Installer	Owns	Why it matters
`install_cli_agent_plugins.sh`	The **entry point**. The **commands** (the `helix` plugin + the per-agent files), the **skills**, and it **chains** the hook installer below.	The one command you run; the one the post-update hook calls automatically.
`install_action_prefix.sh`	The **`UserPromptSubmit` prompt-expansion hook** entry in `<project>/.claude/settings.json` (pointing at `constitution/scripts/hooks/action_prefix_expand.sh` — by reference, never copied). A backup is written to `settings.json.bak` first. It is genuinely idempotent (it refuses to append a hook command it already finds).	It stays the **single owner** of that hook. The `helix` plugin deliberately ships **no** hook: if both the plugin and this installer registered one, every prompt would be **double-expanded**. Commands and prompt-expansion are owned by different installers on purpose — **do not add a hook to the plugin.**

You *may* still run `install_action_prefix.sh` directly (e.g. to re-wire only the hook); it is safe and idempotent. It is simply no longer a required second step.[4]

. How each agent discovers the commands

All four are wired **automatically** by the one install command (§1).

Agent	Discovery path	Source it is linked from
Claude Code	The plugin `helix` from the local marketplace `helix-constitution`. Exposed as `/<name>`, `/helix:<name>`, `/default-<name>`.	`plugins/helix/commands/*.md` — the generator writes them straight into the plugin tree (single source of truth, no copy).
Gemini CLI	`.gemini/commands/*.toml`	`actions/generated/gemini/`
Qwen Code	`.qwen/commands/*.toml`	`actions/generated/qwen/`
Codex CLI	`prompts/*.md`	`actions/generated/codex/`

`link_agent_commands()` creates one **symlink per command file** — BY REFERENCE, never a copy (§11.4.28 / §11.4.80). It **never clobbers a real file**: it replaces only its own symlink or fills a free slot, and WARNs (leaving it untouched) if a hand-written file already sits at that name. Because the links point at the generated files, an **edited expansion** is picked up with no re-link; re-running the installer is only needed when an action is **added or removed**.

The Codex destination is `prompts/` — the convention declared by the repo's own `actions/generated/README.md`.

Honest boundary (§11.4.6). Those three agents have **no pre-submit hook**, so the free-form forms (`BUG: …` , `BUG :: …` , `BUG ---> …`) are handled there by LAYER 1 — the model reading the §11.4.140 block in its context carrier (`GEMINI.md` / `QWEN.md` / `AGENTS.md`) — not by a hook. Only Claude Code gets the deterministic `UserPromptSubmit` expansion.

. Auto-load on constitution pull (§ . .)

`scripts/post_update_hook.sh` is the auto-propagation seam. After a `git pull` / `git submodule update` inside the constitution, run it (or wire it into the project's post-merge hook):

```
bash constitution/scripts/post_update_hook.sh
```

It diffs `ORIG_HEAD..HEAD` (falling back to `HEAD~1`), classifies every changed file, and — when anything under **`actions/**`**, **`plugins/**`**, **`.claude-plugin/**`**, **or `skills/**`** changed — runs `install_cli_agent_plugins.sh` (STEP 4b). So a new directive added upstream appears in your project on the next pull, with no manual step. It also installs skills, merges MCP configs, installs git hooks, and `bash -n`-validates every touched script.[6]

. Adding a NEW directive — a registry row, never code

This is the whole point of the design: **no code changes anywhere.**

1. **Add a row** to `constitution/actions/registry.yaml`:

```yaml
- name: TRIAGE
  version: 1
  namespaces: [DEFAULT]
  slash_bare: auto            # bare /triage honored unless a host built-in collides
  slash_conflicts: []         # declare any known host collision here, as data
  summary: >-
    Triage an incoming report — classify severity, assign a priority, and
    schedule it without starting the fix.
  expansion: >-
    TRIAGE DIRECTIVE — the remainder of this prompt is an item to triage.
    Classify its severity from captured evidence (never a guess), assign a
    priority, and record the decision. Do NOT start the fix.
  rules:
    - "The expansion REPLACES the 'TRIAGE' prefix in any of its forms; the remainder is the
      item."
  composes_with:
    - "11.4.6"
```

Field semantics: REFERENCE.md §3.3.

2. **Regenerate:**

```
bash constitution/scripts/generate_agent_prefix_commands.sh
```

This emits, for the new action: `plugins/helix/commands/triage.md` + `default-triage.md`, `actions/generated/{gemini,qwen}/triage.toml` + `default-triage.toml`, `actions/generated/codex/triage.md` + `default-triage.md`, and it regenerates both READMEs so the inventory cannot drift.

3. **Re-wire** (picks up the new plugin command):

```
bash constitution/scripts/install_cli_agent_plugins.sh
```

The new directive is now live in **every** form on Claude Code — `TRIAGE: …`, `TRIAGE :: …`, `DEFAULT::TRIAGE :: …`, `TRIAGE ---> …`, `/triage …`, `/helix:triage …`, `/default-triage …` — with no code touched anywhere.

. If the new name collides with a host command

Declare it as **data**, do not work around it:

```
slash_conflicts: [triage]
```

The generator then emits an explicit `CONFLICT` comment inside `plugins/helix/commands/triage.md` and a warning row in the plugin README, and the bare `/triage` is left to the host. Users invoke `/helix:triage` or `/default-triage`. The plugin must **never** silently shadow a host command.[7]

. Troubleshooting

A command does not appear in Claude Code

```
bash constitution/scripts/generate_agent_prefix_commands.sh   # regenerate
bash constitution/scripts/install_cli_agent_plugins.sh        # re-install + verify
claude plugin list | grep helix                               # must list the plugin
```

If `claude plugin list` does not show `helix`, the installer will have said so (it verifies, it does not assume). Run the two commands it prints and read the error.

A bare `/name` runs the wrong thing

It collides with a host built-in. That is by design — see §6.1 and USER_GUIDE §5. Use `/helix:<name>` or `/default-<name>`. If the collision is **not yet declared** in the registry, add it to `slash_conflicts:` and regenerate, so the command file and the README warn about it.

A skill does not load

- It must have a **`SKILL.md`** (uppercase). Directories with only a lowercase `skill.md` or only a `register.sh` are **legacy** and are deliberately skipped — the installer reports the skipped count. Four such legacy directories exist (`media-validator`, `scheduled-work-queue`, `session-sync`, `multitrack`); they are not part of this system.
- Check the symlink: `ls -l .claude/skills/`. If a **real directory** (not a symlink) is sitting at that path, the installer refuses to touch it and warns — remove it and re-run.

A command file was left untouched (WARN)

`link_agent_commands()` found a **real file** (not its own symlink) at that name and refused to overwrite it — your hand-written command wins. Rename or remove it and re-run if you want the generated one.

The `claude` CLI is absent

Not an error, and not faked. The installer says so and prints the exact one-time commands:

```
claude plugin marketplace add ./constitution
claude plugin install helix@helix-constitution
```

Everything else still completes — generation, the other agents' command links, the prompt hook, and the skills. Gemini / Qwen / Codex do not need the `claude` CLI at all, and `--check` reports the plugin state as **UNKNOWN** rather than guessing.

Prompts are expanded twice

Something added a `UserPromptSubmit` hook to the plugin, **or** a second hook entry was appended to `.claude/settings.json` by hand. Check:

```
grep -c action_prefix_expand .claude/settings.json   # MUST be exactly 1
```

The plugin must ship **commands only**; the prompt-expansion hook belongs to `install_action_prefix.sh`, which registers it exactly once (§3).

An unregistered `NAME:` line did nothing

Correct. The single-colon form is **registered-action-only**: an unknown token in that form is a NO-OP, because `NOTE:` / `TODO:` / `WARNING:` / `FIXME:` share the shape. The other five forms **ask** on an unknown token. See REFERENCE §4.6.

Sub-system shortcuts do not resolve

They require `python3` **and** `PyYAML`; without them the token falls through to ASK. Behavioural actions are unaffected (they have an awk fallback). An **ambiguous** token (two sub-systems) is deliberately dropped rather than guessed.
[8]

. Fixed defects (changelog)

`post_update_hook.sh` — `detect_changes()` lost every change (FIXED)

Symptom. A constitution pull that changed skills / actions / plugins installed **nothing**, silently. The hook reported "no changes" and exited 0.

Root cause. `detect_changes()` fed its classification loop from a **pipe**:

```
printf '%s\n' "$changed_files" | while IFS= read -r f; do CHANGED_SKILLS+=("$f"); done
```

In bash a `cmd | while …` loop body runs in a **subshell**, so every `+=` was discarded when that subshell exited. The caller then saw **empty arrays** (reproduced probe: `AFTER-LOOP count = 0`).

Fix. Process substitution keeps the loop in the current shell:

```
done < <(printf '%s\n' "$changed_files")
```

The script now carries a comment at that exact line explaining why it must never be re-piped (§11.4.201 — a guard must assert the REAL condition; a hook that silently installs nothing is a FAIL-open bluff).

Verify the fix on your own tree by changing a registry row, committing inside the constitution, and running `bash constitution/scripts/post_update_hook.sh` — it must report `1 action/plugin file(s) changed` and run STEP 4b.[9]

. Uninstall

Nothing is copied into the project, so removal is just unlinking:

```
# 1. the plugin
claude plugin uninstall helix@helix-constitution
claude plugin marketplace remove helix-constitution

# 2. the skills (symlinks only — the sources stay in the constitution)
rm -f .claude/skills/action-prefix-system \
      .claude/skills/reporting-workable-items \
      .claude/skills/workable-item-lifecycle

# 3. the prompt-expansion hook — remove the UserPromptSubmit entry that references
#    constitution/scripts/hooks/action_prefix_expand.sh
#    (a backup from install time is at .claude/settings.json.bak)
$EDITOR .claude/settings.json

# 4. the other agents' commands (symlinks the installer created — §4)
find .gemini/commands .qwen/commands prompts -maxdepth 1 -type l -delete 2>/dev/null
```

Deleting only symlinks (`-type l`) leaves any hand-written command file of your own in place.

The constitution submodule itself is untouched — the generated files under `actions/generated/` and `plugins/helix/commands/` are regenerated artifacts and can be left in place.